Abstraction in Java

Introduction

Abstraction: Abstraction is the process of Hiding implementation details and showing only essential features. and Showing essentials.

Abstraction focuses on what an object does, not how it does.

Why we use Abstraction:

- Reduce complexity
- Increase security
- Improve flexibility
- Support loose coupling
- Focus on what, not how

Achieved usin
abstract clas
and interface

Very importa
OOP concept

Car → Steering, Brake, Accelerator
(User uses features, not internal engine)

```
abstract class Shape {
   abstract void draw();
}
class Circle extends Shape {
   void draw() {
      System.out.println("Drawing Circle");
   }
}
```

- abstract class
cannot be instan
- abstract meth
has no body

Abstraction hides internal details and shows only functione

Abstract Class in Java

Abstract Class:

A class declared using the abstract keyword is called abstract class.

Cannot be instantiated

Features of Abstract Class:

- Can have abstract and non-abstract methods
- Can have constructors
- Can have variables
- Cannot create object directly No object creation

Syntax

```
abstract class ClassName {
    abstract void method1();
    void method2() { }
}
```

```
abstract class Animal {
    abstract void sound();
}
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
```

- Child must implemen abstract methods
- Abstract class provid partial abstraction

Can have implemented methods

Abstract class provides partial abstraction

Abstract Methods in Java

Abstract Method:

A method declared without body and with abstract keyword is called abstract method.

Rules of Abstract Method:

- Declared using abstract keyword
- Cannot have body
- Must be implemented by child class
- Cannot be final or static
- Must be overridden

Syntax

```
abstract void show();
```

```
abstract class Shape {
  abstract void draw();
}

class Circle extends Shape {
  void draw() {
    System.out.println("Drawing Circle");
  }
}
```

Explanation:

- Child class provi
  implementation
- Supports runtim
  polymorphism

Cannot create ob
of abstract class

Abstract method
force implemento

Abstract methods define what to do, not how

Abstract Class vs Interface

Abstract Class:

- Can have abstract and concrete methods
- Can have constructors
- Can have instance variables
- Supports partial abstraction

Interface:

- Methods are abstract by default
- Cannot have constructors
- Variables are public static final
- Supports Full abstraction

Feature	Abstract Class	Interface
Methods	Abstract + Non	Abstract only
Variables	Any type	public static final
Constructors	Yes	No
Multiple inheritance	No	Yes
Abstraction level	Partial	Full

Interface use
100% abstrac

- Use abstract class for base behavior
- Use interface for multiple inheritance

Very importa
exam questi

Choose abstract class or interface based on requirement

Why Abstraction is Needed

Why we need Abstraction:

1. Reduce Complexity
   - Hides unnecessary details
   - Shows only required features

Hide complexity

2. Improve Security
   - Internal logic is hidden
   - Only interface is exposed

3. Increase Flexibility
   - Code can change internally
   - Without affecting user

Flexible design

4. Support Loose Coupling
   - Changes in one class
   - Do not affect others

5. Better Maintenance
   - Code is easy to update
   - Easy debugging

Abstraction simplifies syst

Simple real-life example:
Remote → Buttons only
(User doesn't see circuits)

Very importar theory questi

Abstraction hides details and exposes functionali

Abstraction using Interface

Interface:

An interface is a blueprint that contains only abstract methods and constants.

Features of Interface:

- Supports full abstraction
- Supports multiple inheritance
- All methods are public & abstract by defau
- Variables are public static final

Syntax

```
interface Shape {
   void draw();
}
class Circle implements Shape {
   public void draw() {
      System.out.println("Drawing Circle");
   }
}
```

- implements keyword is used
- Child class must implement all methods

Interface achieves 1 abstracti

Used for loose coup

Interface is used to achieve full abstraction

Multiple Inheritance using Interface

Multiple Inheritance:

When a class implements More than one interface, it is called Multiple inheritance.

Why Java allows this:

- No ambiguity
- Only method declarations
- No method body in interfaces

```
interface A {
    void show();
}
interface B {
    void display();
}
class Demo implements A, B {
    public void show() {
        System.out.println("Show");
    }
    public void display() {
        System.out.println("Display");
    }
}
```

- Class can implement multiple interfaces
- Avoids diamond problem

Java suppor multipl inherit via inte

Very common exam questio

Interfaces enable safe multiple inheritance

default & static Methods in Interface

Before Java 8:
- Interfaces had only abstract methods

After Java 8:
- Interfaces can have default and static methods

Java 8 feature

default Method:
- Has method body
- Can be overridden

static Method:
- Has method body
- Cannot be overridden
- Called using interface name

```
interface Test {
    default void show() {
        System.out.println("Default method");
    }
    static void display() {
        System.out.println("Static method");
    }
}
class Demo implements Test { }
```

Asked i
intervie

Importa
Java 8
update

Method calling:

```
Demo d = new Demo();
d.show();
Test.display();
```

- default gives backward compatibility
- static belongs to interface

Interfaces can have default and static methods

Functional Interface & Lambda (Intro)

Functional Interface:

An interface that contains only One abstract method is called Functional interface.

Example:

```
interface Test {
    void show();
}
```

Lambda Expression:

A Short syntax a short way to implement a Lambda functional interface.

Example comparison:

Without Lambda:

```
Test t = new Test() {
    public void show() {
        System.out.println("Hello");
    }
};
```

With Lambda:

```
Test t = () -> {
    System.out.println("Hello");
};
```

- Java 8 feature
- Used in streams

- Reduces boilerplate code
- Makes code concise

Lambda provides compact implementation of interface.

Abstraction – Real Use Cases & Benefits

Real Use Cases:

- Payment systems (UPI, Card, Cash)
- Shapes (Circle, Rectangle, Square)
- Remote controls
- Banking systems

Different implementations, same interface

Benefits of Abstraction:

- Hides complexity
- Improves security
- Supports loose coupling and Flexible design
- Increases flexibility
- Easy to extend

```
interface Payment {
    void pay();
}
class UPI implements Payment {
    public void pay() {
        System.out.println("Paid using UPI");
    }
}
class Card implements Payment {
    public void pay() {
        System.out.println("Paid using Card");
    }
}
```

Very commo
intervie
topic

Used i
framew

- Same interface, different behavior
- Easy to add new payment types

Abstraction makes system flexible and extensible

Abstraction – Summary & Key Points

Core Concept:

Abstraction hides implementation and shows only essential features. Shows essentials.

Ways to achieve Abstraction:

- Abstract class
- Interface

Abstract Class:

Write definiti
first in exam

- Partial abstraction
- Can have concrete methods
- Can have constructors

Interface:

- Full abstraction
- Supports multiple inheritance
- All methods public & abstract

Important Keywords:

Use keywo
for marks

- abstract
- interface
- implements
- extends
- default
- static

Common Mistakes:

- Trying to create object of abstract class
- Forgetting to implement methods
- Confusing abstract class with interface

Abstraction in Java – Final Revision

Quick Revision:

- Abstraction hides implementation details Hide deta
- Shows only essential features Show essentials
- Achieved using abstract class & interface
- Focus on what, not how

Important Keywords (boxed neatly):

abstraction abstract interface implements
extends default static

MCQ / Viva Important Points:

- Abstract class cannot be instantiated
- Abstract methods have no body
- Interface supports multiple inheritance
- Child must implement all abstract methods
- static methods in interface cannot be overridden
- default methods can be overridden

True / False:

- Interface can have constructors (False)
- Abstract class can have non-abstract methods (Tru
- Lambda works with functional interface (True)

One-line Answers:

- Abstraction hides internal logic
- Interface gives 100% abstraction

Abstraction simplifies complex systems in Java